실습 가이드

▸ 본 프로젝트에서 쓰일 설정값 초기 정보(참고)

1️⃣ Bastion 호스트 접속

1. AWS 콘솔 → EC2 → Bastion 인스턴스 → 퍼블릭 IP 확인
2. 로컬 PC에서 접속:

▸ bastion에서 web 접속할 때 bastion에 키가 없어서 web에 접속 안될 수 있음

```
ssh -i mykey.pem ubuntu@<bastion-public-ip>
```

1. 접속 성공 확인: `whoami` → `ubuntu`

2️⃣ Web 서버 접속 (프라이빗 서브넷)

EC2 별도로 생성해서 접속함(굳이 만들어진 거로 접속하지 않아도 돼서)

▼ EC2 생성

- **AMI**: Ubuntu 20.04 LTS (권장)
- **Instance type**: t2.micro (프리티어)
- **VPC**: 이미 구성된 VPC 선택
- **Subnet**: 프라이빗 서브넷(예: `10.0.20.0/24` in 1a, `10.0.30.0/24` in 1c)
- **Auto-assign Public IP**: Disable (끄기)
- **Security Group**: 기존 `web-sg` 선택 (ALB→Web, Bastion→Web 통신 허용됨)
- **Key Pair**: 기존 Bastion에서 사용하는 것과 동일한 key.pem

- **Key Pair**: 기존 Bastion에서 사용하는 것과 동일한 key.pem

*bastion-key가 로컬 pc에만 위치해있어 로컬 PC에 있는 키파일을 Bastion 서버로 옮겨주는 과정이 필요했음. bastion을 통해 웹으로 접속하므로 bastion 서버에서도 bation 키 파일이 필요했음.

1. Bastion에서 Web 서버 내부 IP로 접속:

```
ssh -i bastion-key.pem ubuntu@<web-private-ip>
```

2. 접속 성공 후 `hostname` 으로 Web 서버임 확인

---

*하나의 서버 안 2개의 컨테이너

1. **Flask**
   - Python 기반 웹 애플리케이션 서버 (WSGI 앱)
   - 비즈니스 로직 처리 (예: `app.py`)
   - 외부 요청을 직접 받기보다는, 보통 Nginx 뒤에서 동작
2. **Nginx**
   - 리버스 프록시 / 정적 파일 서빙 / 로드밸런싱 역할
   - 외부 클라이언트 요청을 받아서 Flask 컨테이너로 전달

즉, 같은 EC2 안에서 **Flask 컨테이너** + **Nginx 컨테이너**가 함께 동작하는 구조예요.

- 외부 → ALB → Nginx → Flask → 응답 반환

3️⃣ Nginx 웹 서버 설치 및 ALB 연결

1. Nginx 설치(웹 서버에 접속한 상태):

```
sudo apt update
sudo apt install nginx -y
```

2. Nginx 실행 확인:

```
curl http://localhost
```

→ "Welcome to nginx!" 나오면 성공

3. ALB 생성 후 Target Group에 Web 서버 등록

2가지 시나리오가 있음

1. 인프라 담당자가 만든 ALB에 Web 서버를 Target Group으로 붙인다.

▶ 토글

2. 각자가 직접 ALB를 만들어서 실습해본다.

▶ ALB(Application Load Balancer) 생성

4. 브라우저에서 ALB DNS로 접속 → Nginx 페이지 확인

▼ ALB(Application Load Balancer) 생성

- **EC2 → Load Balancers → Create Load Balancer → Application Load Balancer 선택**
  - 이름: `project-alb`
  - Scheme: Internet-facing
  - IP type: IPv4
  - VPC: 이미 구성된 VPC 선택
  - AZ: Web 서버가 들어 있는 Subnet 두 개 선택 (예: 1a, 1c)

→test-ec2, test-ec2-1 인스턴스 2개 만들어서 web-tg1의 대상그룹에 등록함



▶ web 서버 하나가 unhealthy인 상황

→ 기존에 만들어져있는 프라이빗 라우팅 테이블 사용하려다가 결국에는 라우팅 테이블 새로 만들고 private subnet 다 연결해서 결국 nat-gateway에 연결시킴(하지만 아직 unhealthy) (원래는 sudo apt update도 안됐는데 연결시키고 됨)

🛠️ 해결 방법

해당 인스턴스에 Nginx 설치 후 실행하면 됩니다.

1. 업데이트 & Nginx 설치

bash 복사 편집

```
sudo apt update
sudo apt install nginx -y
```

2. 실행 & 부팅 시 자동 실행 설정

bash 복사 편집

```
sudo systemctl start nginx
sudo systemctl enable nginx
```

3. 확인

bash 복사 편집

```
systemctl status nginx
curl http://localhost
```

→ "Welcome to nginx!" 페이지가 나오면 정상입니다.

4. 이제 다시 AWS 콘솔 → **Target Group → Targets** 들어가서 상태를 보면,
   몇 분 후 `unhealthy` → `healthy` 로 바뀔 거예요.

→조금 지나니 healthy 됌



4. 브라우저에서 ALB DNS로 접속 → Nginx 페이지 확인

test-alb-1647381146.ap-northeast-1.elb.amazonaws.com

nginx에 오신 것을 환영합니다!

이 페이지가 표시되면 nginx 웹 서버가 성공적으로 설치되어 작동 중인 것입니다. 추가 구성이 필요합니다.

온라인 문서 및 지원은 nginx.org 를 참조하세요 . 상용 지원은 nginx.com 에서 제공됩니다 .

nginx를 사용해 주셔서 감사합니다.

4 Flask 앱 배포(두 서버 모두)

1. Python & Flask 설치:

```
sudo apt install python3-pip -y
pip3 install flask
```

2. 앱 작성:

```
nano app.py
```

```
from flask import Flask
app = Flask(__name__)

@app.route("/")
def hello():
    return "Hello from Flask on AWS!"

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

3. 실행:

```
python3 app.py
```

1. test-ec2

```
(venv) ubuntu@ip-10-0-21-119:~$ ls
app.py  venv
(venv) ubuntu@ip-10-0-21-119:~$ python app.py
 * Serving Flask app 'app'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment.
 Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://10.0.21.119:5000
Press CTRL+C to quit
```

2. test-ec2-1

```
(venv) ubuntu@ip-10-0-20-225:~$ ls
app.py  venv
(venv) ubuntu@ip-10-0-20-225:~$ python app.py
 * Serving Flask app 'app'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment.
 Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://10.0.20.225:5000
Press CTRL+C to quit
```

4. 보안 그룹에 **5000 포트 허용** → ALB에서 접속 확인(간단 과정이고 보안상 문제로 안함)
5. 📌 실무형 Flask 배포: 단계별 안내

▶ 🔹 실무형 Flask 배포 (2대 Web 서버 공통)

5️⃣ UFW 방화벽 실습

🟢실무형 UFW 설정 (Flask 배포 완료 기준)

1. UFW 설치

```
sudo apt update
sudo apt install ufw -y
```

👉 OS 차원의 방화벽 도구 설치. (SG와 함께 "이중 방어" 목적)

2. 기본 정책 설정

Bash 복사 캡션 ···

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
```

- **의미**:
  - 외부에서 들어오는 건 모두 차단 (보안 ↑)
  - 서버에서 나가는 건 허용 (패키지 설치, 로그 전송, API 호출 가능)

3. 필요한 포트만 열기

실무형 구조에서는 Flask(5000)를 외부에서 직접 열 필요가 없습니다.

Flask는 **내부에서만 (127.0.0.1:5000)** Nginx가 접근하기 때문이에요.

따라서 열어야 할 포트는:

- `22` : SSH (Bastion에서 접근)
- `80` : HTTP (ALB → Nginx)

```
sudo ufw allow 22     # SSH (Bastion에서만 접근)
sudo ufw allow 80     # HTTP (ALB 연결용)
```

❌ `5000` 은 이번엔 아예 열지 않습니다.

(이미 Nginx 프록시를 통해 ALB에서 Flask로 트래픽이 안전하게 전달되니까요.)

---

4. 활성화

```
sudo ufw enable
```

👉 SSH 끊길 수 있다는 경고 나오면 `y` 입력.

6. 테스트

1. **Bastion → Web 서버 SSH (22)**
   - 정상 접속 ✅
2. **브라우저 → ALB DNS (80)**
   - Flask 메시지 `"Hello from Flask on AWS!"` 확인 ✅
3. **브라우저 → Web 서버 퍼블릭 IP:5000**
   - ❌ 차단 (정상, 실무형 구조에서는 막혀야 함)

▼ 🔎 각 단계별 테스트 방법(구체적으로)

1️⃣ Bastion으로 접속

내 로컬 PC (깃 Bash, 터미널 등)에서:

```
ssh -i mykey.pem ubuntu@<Bastion-Public-IP>
```

✅ Bastion 서버 로그인 완료.

2️⃣ Bastion에서 Web 서버 접속

Bastion에 들어온 뒤, 다시 **Web 서버 Private IP**로 SSH:

```
ssh ubuntu@<Web-Server-Private-IP>
```

✅ 여기까지 되면 **UFW 22번 규칙 + Security Group** 이 정상적으로 적용된 것.

3️⃣ ALB 테스트 (브라우저)

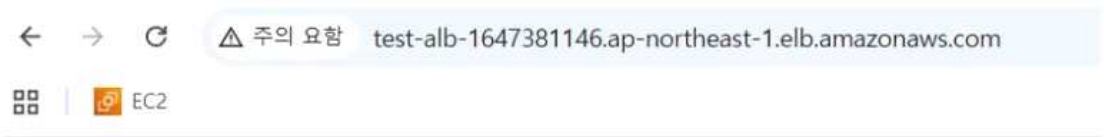
- 이건 **내 로컬 PC 브라우저**에서 바로 해요.
- 주소창에 입력:

```
http://<ALB-DNS>
```

✅ "Hello from Flask on AWS!" 가 떠야 정상.

(ALB가 Web 서버의 80 포트까지 잘 전달하는지 확인하는 과정)

→새로고침하면 번갈아 두 문구가 나옴(반복해서 실행하면 ALB가 Web1 → Web2로 요청을 분배하는 게 확인)



AWS Webserver1의 Flask에서 안녕하세요!



AWS WebServer2의 Flask에서 안녕하세요!

6 Auto Scaling 구성

▼ Auto Scaling & ALB 기반 확장 실습 단계

1 AMI 생성

- **작업**
  1. EC2 대시보드 → 현재 Web 서버 인스턴스 선택
  2. [작업] → [이미지 및 템플릿] → [이미지 생성]
  3. 이름: `web-server-golden-image`
  4. 불필요한 EBS 스냅샷 옵션 해제 후 생성
- **왜 필요?**

  → 현재 Web 서버(Flask+Nginx+UFW 설정 포함)를 "표준 이미지"로 저장해야, Auto Scaling에서 동일한 서버를 자동으로 찍어낼 수 있음.



▶ 골든 AMI란?

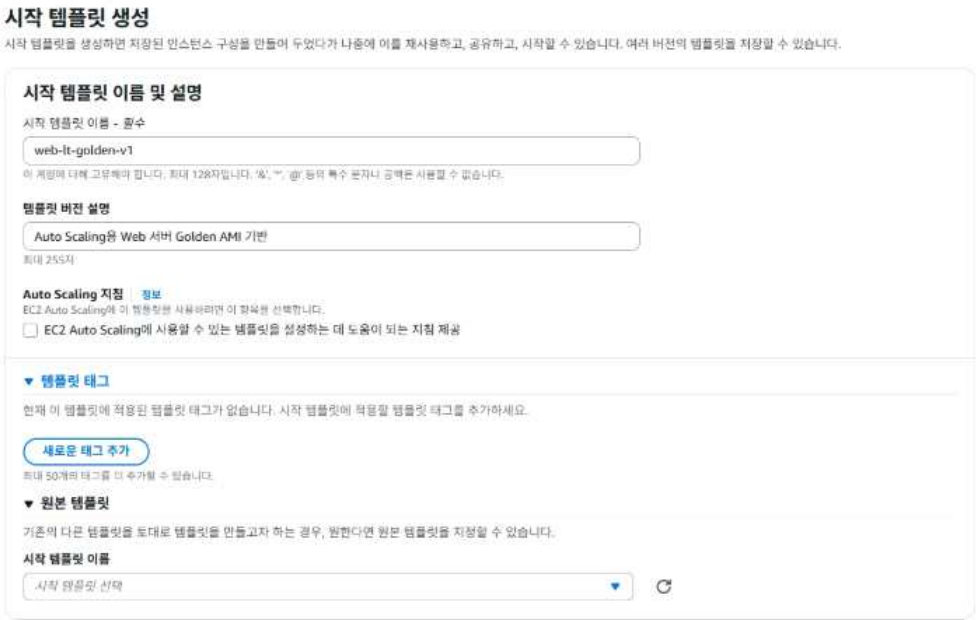
2 Launch Template 생성

- **작업**
  1. EC2 대시보드 → [시작 템플릿] → 새 템플릿 생성
  2. AMI: 방금 만든 `web-server-golden-image` 선택
  3. 인스턴스 유형: `t3.micro` (프로젝트 맞게 선택)
  4. 네트워크: 기존 VPC + 프라이빗 서브넷
  5. 보안 그룹: Web 서버 SG (22, 80만 허용된 것)
  6. User Data: (추가 스크립트 불필요 — 이미 AMI에 반영되어 있음)
- **왜 필요?**

  → Auto Scaling은 "런치 템플릿"을 기준으로 새 인스턴스를 띄움.

  → 우리가 만든 AMI + 네트워크/보안 설정을 하나의 템플릿으로 묶어두는 단계.

  ▼ 생성 과정 이미지



3️⃣ Auto Scaling Group 생성

- **작업**
  1. [Auto Scaling 그룹] → 생성
  2. Launch Template: 위에서 만든 것 선택
  3. VPC/Subnet: Web 서버 프라이빗 서브넷 선택
  4. Target Group: 기존 ALB Target Group 연결
  5. Desired Capacity: 2
  6. Min: 1, Max: 3
  7. Scaling Policy: CPU 70% 이상 → +1, 30% 이하 → -1

  ▶ 📌 전제 (프로젝트 조건 정리)

- **왜 필요?**

  → 트래픽이 몰리면 자동으로 서버가 늘어나고, 한가하면 줄어드는 구조 완성.

  → "실무 Auto Scaling"을 체험하는 핵심 단계.

  ▼ Auto Scaling Group 생성 과정 이미지



Docker 기반 버전(지금까지의 과정은 EC2 Systemd 버전)

Nginx **컨테이너** + Flask **컨테이너**를 docker compose로 올리고, 그 상태를 AMI로 베이크해서 ASG로 확장)

▸ systemd vs docker

🚀 베이크 전용 웹서버 세팅 절차 (B안)

→하나의 웹서버에서만 작업하면 됨

▸ **베이크 서버에 대해**

Bastion → 웹서버(web-1a) 접속 후, 아래 순서대로 복붙

0. 선행 정리 (잔재 서비스/포트 제거)

```
sudo systemctl stop nginx 2>/dev/null || true
sudo systemctl disable nginx 2>/dev/null || true

sudo systemctl stop flask 2>/dev/null || true
sudo systemctl disable flask 2>/dev/null || true

sudo systemctl stop app-compose 2>/dev/null || true
sudo systemctl disable app-compose 2>/dev/null || true

sudo ss -ltnp | egrep ':80 |:5000 ' || echo "80/5000 free"
```

2. 앱/프록시 자산 배치

2-1. Flask 앱 (헬스체크용, 디버그용)

```
tee ./app.py >/dev/null <<'PY'
from flask import Flask, jsonify
app = Flask(__name__)

@app.route("/")
def hello():
    return "Hello from Flask on AWS!", 200

@app.route("/healthz")
def healthz():
    return jsonify(status="ok"), 200
PY
```

👉 **의미**: 단순 Flask 앱, `/healthz` 는 ALB 헬스체크용

2-2. Nginx 설정

```
tee ./docker/nginx/nginx.conf >/dev/null <<'NGINX'
server {
  listen 80;
  server_name _;

  location / {
    proxy_pass http://app:5000;
    proxy_set_header Host $host;
```

3. Docker & Compose 설치 (한 번만)

```
set -eu
sudo apt update
sudo apt install -y ca-certificates curl gnupg

# Docker 공식 저장소 추가
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
 | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
echo \
  "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] \
  https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo $VERSION_CODENAM
E) stable" \
 | sudo tee /etc/apt/sources.list.d/docker.list >/dev/null

sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-com
pose-plugin
sudo systemctl enable --now docker
# (선택) sudo 없이 docker 쓰려면:
sudo usermod -aG docker ubuntu || true
```

설치 확인(빠른 체크)

```
docker --version
docker compose version
systemctl is-active docker     # active 가 나오면 OK
# (인터넷 통신 가능하면) 기본 테스트:
docker run --rm hello-world
```

4) 컨테이너 기동 & 검증(pull → up → 검증)

```
cd /opt/app

# 1) 이미지 가져오기 (무결성/자격 확인)
docker compose pull

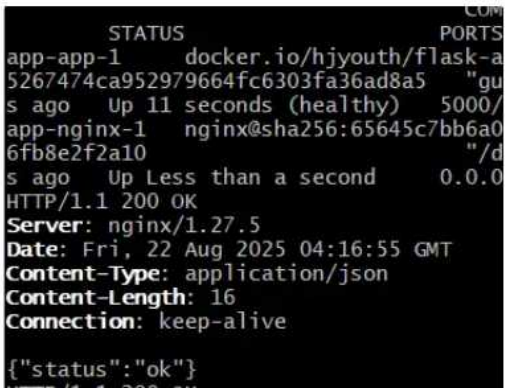
# 2) 서비스 기동
docker compose up -d

# 3) 상태 테이블
docker compose ps

# 4) 애플리케이션/헬스 확인
curl -i http://localhost/healthz      # 기대: HTTP/1.1 200 OK
curl -i http://localhost/             # 기대: "Hello from Flask on AWS!"
```

의미:

- pull → up -d로 베이크 서버에서 표준 상태를 만든 뒤, 이 상태를 **AMI로 굽는** 게 목표.



5) 부팅 자동 실행(systemd) — 아직 안 했다면

```
sudo tee /etc/systemd/system/app-compose.service >/dev/null <<'EOF'
[Unit]
Description=Docker Compose for Nginx+Flask
After=network-online.target docker.service
Wants=docker.service

[Service]
Type=oneshot
WorkingDirectory=/opt/app
RemainAfterExit=true
ExecStart=/usr/bin/docker compose up -d
ExecStop=/usr/bin/docker compose down
TimeoutStartSec=0

[Install]
WantedBy=multi-user.target
EOF

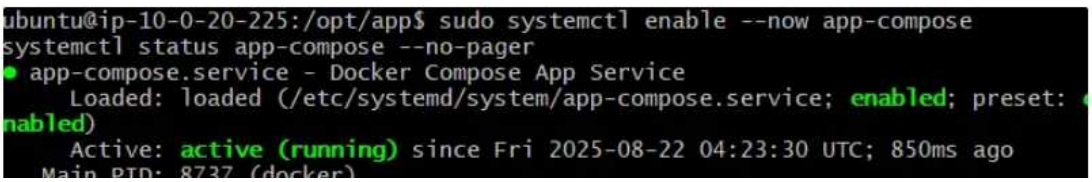
sudo systemctl daemon-reload
sudo systemctl enable --now app-compose
systemctl status app-compose --no-pager
```

의미:

- **재부팅 후 자동 기동** 보장 → ASG로 확장/치유 시 **셀프 힐링**.

▶ 부팅 자동실행했더니 오류

성공 이미



7) AMI 베이크 → LT/ASG 반영 → Instance Refresh

```
# (선택) 새 인스턴스 기동을 빠르게 하려면 캐시 미리 채우기
docker pull "$(grep ^APP_IMAGE= /opt/app/.env | cut -d= -f2)"
docker pull "$(grep ^NGINX_IMAGE= /opt/app/.env | cut -d= -f2)"

# 이후 콘솔 권장 절차
# 1) 이 베이크 서버를 AMI로 이미지화(버전 표기)
# 2) Launch Template 새 버전에 해당 AMI 지정
# 3) ASG에 적용 → Instance Refresh 실행
#    - MinHealthyPercentage = 90%
#    - InstanceWarmup = 90~120s
#    - Health Check Type = ELB
```

의미:

- **"한 대에서 정답 만들고 → AMI로 표준화 → ASG 롤링 반영"** 패턴 완성.
- 확장/복구 시 **빌드 없이 pull만** 하므로 빠르고 일관됨

```
ubuntu@ip-10-0-20-225:~$ curl -I http://localhost/healthz
HTTP/1.1 200 OK
Server: nginx/1.27.5
Date: Fri, 22 Aug 2025 06:04:18 GMT
Content-Type: application/json
Content-Length: 16
Connection: keep-alive
```